Week 1 Lab 1 — Numpy Refresher

26th October 2026

1 NumPy Refresher

Welcome to the Numpy refresher lab session! NumPy is a powerful linear algebra library for Python. What makes it so important is that almost all of the libraries in the PyData ecosystem (pandas, scipy, scikit-learn, etc.) rely on NumPy as one of their main building blocks. This lab will provide a brief overview of some of the most commonly used Numpy functions and features, including array creation and some common operations and methods. By the end of this lab, you will have an understanding of how to use Numpy to perform a wide range of operations on arrays and matrices in Python. Let's get started!

1. Download the Numpy refresher lab file from the provided link below.
2. Open Visual Studio Code, and go to File > Open Folder. Select the folder where you have downloaded the lab files or directly double click the .ipynb file to open in your default environment like VS code.
3. You should now be able to run and practice the code provided in the notebook.
4. You can now open the lab notebook and start practicing the numpy operations.

1.1 *Note*: Numpy Installation Instructions

NumPy should be already included in your environment! that is if you have installed python using Anaconda distribution.

1.2 Before you begin : Importing NumPy

Once you've installed NumPy you can import it as a library:

[1]:
```
import numpy as np
```

NumPy has many built-in functions and capabilities. Covering all of them will be out of the scope for this course, so we will focus on some important aspects of NumPy: vectors, arrays, matrices and number generation.

2 1. NumPy Arrays

NumPy arrays come in two forms : vectors and matrices. Vectors are strictly 1-dimensional (1D) arrays and matrices are 2D (but you should note a matrix can still have only one row or one

column).

Why use Numpy array? Why not just a list? *Please revise your python basics before you start understanding these libraries and their concepts.

There are lot's of reasons to use a Numpy array instead of a "standard" python list object. Our main reasons are: * Memory Efficiency of Numpy Array when compared to python lists * Easily expnadable to n-dimensions * Speed of calculations of numpy array* * Numpy is an integral library for all the data science and machine learning projects

2.1 Task 1.1 Simple examples of the use of Numpy arrays

[2]:
```
my_list = [1,2,3]
my_array = np.array([1,2,3])
```

[3]:
```
type(my_list)
```

[3]:
```
list
```

[4]:
```
type(my_array)
```

[4]:
```
numpy.ndarray
```

[5]:
```
my_list
```

[5]:
```
[1, 2, 3]
```

[6]:
```
my_array
```

[6]:
```
array([1, 2, 3])
```

[7]:
```
my_matrix = [[1,2,3],[4,5,6],[7,8,9]]
my_matrix
```

[7]:
```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

[8]:
```
np.array(my_matrix)
```

[8]:
```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

Alternatively, to display the matrix in a more readable format, we can use the Pandas library. Pandas is a powerful data manipulation library that provides data structures like DataFrames, which are great for displaying tabular data.

[9]:
```
import pandas as pd
display(pd.DataFrame(my_matrix))
```

```
   0  1  2
0  1  2  3
1  4  5  6
2  7  8  9
```

2.2 Task 1.2 Built-in Methods to create arrays

There are lots of built-in ways to generate arrays.

2.2.1 arange

Return evenly spaced values within a given interval. [reference — deprecated]. Check the range carefully for inclusion and exclusion. updated reference

`arange` can be called with a varying number of positional arguments:

- `arange(stop)`: Values are generated within the half-open interval `[0, stop)` (in other words, the interval including start but excluding stop).
- `arange(start, stop)`: Values are generated within the half-open interval `[start, stop)`.
- `arange(start, stop, step)`: Values are generated within the half-open interval `[start, stop)`, with spacing between values given by step.

[10]:
```
np.arange(10,20)
```

[10]:
```
array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
```

[11]:
```
np.arange(10,20,2)
```

[11]:
```
array([10, 12, 14, 16, 18])
```

2.2.2 Task 1.3 zeros and ones

Generate arrays of zeros or ones. [reference — deprecated]

updated reference

[12]:
```
np.zeros(5)
```

[12]:
```
array([0., 0., 0., 0., 0.])
```

The first argument to `np.zeros` is the shape of the array. If you want a 2D array, you can pass a tuple as the shape.

[13]:
```
np.zeros((3,4))
```

[13]:
```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

[14]:
```
np.ones(5)
```

[14]:
```
array([1., 1., 1., 1., 1.])
```

[15]:
```
np.ones((3,4))
```

[15]:
```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

2.2.3 Task 1.4 linspace

Return evenly spaced numbers over a specified interval. [reference]

`numpy.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0, *, device=None)`

Parameters: - start: array_like

The starting value of the sequence. - stop: array_like

The end value of the sequence, unless endpoint is set to False. In that case, the sequence consists of all but the last of num + 1 evenly spaced samples, so that stop is excluded. Note that the step size changes when endpoint is False. - num: int, optional

Number of samples to generate. Default is 50. Must be non-negative.

[16]:
```
np.linspace(0,15,4)
```

[16]:
```
array([ 0.,  5., 10., 15.])
```

2.2.4 Task 1.5 eye

Creates an identity matrix (research if you do not know what an identity matrix is) [reference — deprecated]

updated reference

`numpy.eye(N, M=None, k=0, dtype=<class 'float'>, order='C', *, device=None, like=None)`

An identity matrix is a square matrix of size n × n that has ones (1) along its main diagonal (from top-left to bottom-right) and zeros (0) everywhere else.

[17]:
```
np.eye(5)
```

[17]:
```
array([[1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 1.]])
```

2.3 Task 1.6 Random

With Numpy you can create random number arrays:

2.3.1 rand

Creates an array of the given shape and populates it with random samples from a uniform distribution over `[0, 1)`. [reference — deprecated]

updated reference

`random.rand(d0, d1, ..., dn)`

Creates an array of the given shape and populates it with random samples from a uniform distribution over `[0, 1)`.

[18]:
```
np.random.rand(3)
```

[18]:
```
array([0.28759415, 0.43318978, 0.02585206])
```

[43]:
```
values = np.random.rand(4, 4)

print(f"Type of values: {type(values)}")
print(values)
print('Average:', values.mean())
```

```
Type of values: <class 'numpy.ndarray'>
[[0.24989901 0.9176299  0.26414685 0.71777369]
 [0.86571503 0.80707948 0.21055058 0.16724303]
 [0.04670639 0.03942231 0.20023081 0.9985434 ]
 [0.37278698 0.76051027 0.47347444 0.50971531]]
Average: 0.4750892187308924
```

Example with more dimensions:

[20]:
```
np.random.rand(4,4,4)
```

[20]:
```
array([[[6.08115201e-01, 6.72493135e-01, 4.15501830e-01, 8.76408328e-01],
        [4.04571997e-01, 4.22743172e-01, 8.66403328e-03, 9.83993101e-01],
        [8.05182557e-01, 7.52697636e-05, 1.75159108e-01, 7.48046308e-01],
        [6.21875237e-01, 4.84525107e-01, 6.92367093e-01, 5.75580951e-01]],

       [[9.51336854e-01, 1.97990525e-01, 8.95843000e-02, 6.83036286e-01],
        [8.75067596e-01, 8.04034004e-01, 9.52734116e-01, 7.80035336e-01],
        [3.27695157e-01, 9.29368326e-03, 3.72156458e-01, 3.01978627e-01],
        [8.23109180e-01, 3.06305640e-01, 5.65112573e-02, 2.50147887e-01]],

       [[9.23441300e-01, 3.78429404e-02, 3.21879825e-01, 2.66705204e-01],
        [2.05007003e-01, 7.84717800e-01, 1.73110679e-01, 1.86485486e-01],
        [8.52556094e-02, 9.44267108e-01, 6.90471861e-01, 5.00787813e-01],
        [6.53579743e-01, 6.66958849e-02, 8.47721920e-01, 5.01714670e-01]],

       [[6.82298979e-01, 4.52962326e-02, 5.36918594e-01, 7.52549852e-01],
        [6.99670465e-01, 9.96047952e-01, 8.89967105e-01, 3.00650474e-01],
        [8.92099749e-01, 7.93931468e-02, 1.97150411e-01, 1.38547330e-01],
```

```
       [7.50690696e-01, 4.32743593e-01, 4.85502247e-01, 9.11356989e-01]]])
```

2.3.2 randn

Returns a sample (or samples) from the "standard normal" distribution [$= 1$]. Unlike **rand** which is uniform, values closer to zero are more likely to appear. [reference — deprecated]

updated reference

[21]:
```
np.random.randn(3)
```

[21]:
```
array([-0.89146711, -0.9017733 ,  1.63531722])
```

[46]:
```
values = np.random.randn(4,4)

#print(f"Type of values: {type(values)}")
print(f"Type of values: {type(values).__module__}.{type(values).__name__}")
print(values)
print(f"Shape of values: {values.shape}")
print(f"Size of values: {values.size}") # the ndarray.size attribute returns␣
 ↪the total number of elements in an array
print(f"Average of values: {values.mean()}")
```

```
Type of values: numpy.ndarray
[[ 1.3970324  -2.76425349 -0.72005224  1.77887671]
 [ 1.18321193  0.37804815  0.9846353  -0.1221545 ]
 [-0.30802918 -0.21904899 -1.23574468  0.60220596]
 [ 0.80996824  0.0915304  -1.08673324 -1.04236655]]
Shape of values: (4, 4)
Size of values: 16
Average of values: -0.0170546114551082
```

Comparing `rand` and `randn`:

[23]:
```
print('rand average:', np.random.rand(1000, 1000).mean())
print('randn average:', np.random.randn(1000, 1000).mean())
```

```
rand average: 0.4995700578464567
randn average: -7.627123215352327e-06
```

2.3.3 randint

Returns random integers from `low` (inclusive) to `high` (exclusive). [reference — deprecated]

updated reference

`random.randint(low, high=None, size=None, dtype=int)` Return random integers from low (inclusive) to high (exclusive).

Return random integers from the "discrete uniform" distribution of the specified dtype in the "half-open" interval [low, high). If high is None (the default), then results are from [0, low).

[24]:
```
np.random.randint(1,10)
```

[24]:
```
4
```

[25]:
```
np.random.randint(1,10,5)
```

[25]:
```
array([5, 7, 7, 1, 9])
```

2.3.4 seed

Can be used to set the random state, so that the same "random" results can be reproduced. [reference — deprecated]

updated reference This is a convenience, **legacy function** that exists to support older code that uses the singleton RandomState. Best practice is to use a dedicated `Generator` instance rather than the random variate generation methods exposed directly in the random module.

[50]:
```
np.random.seed(15)
np.random.rand(3)
```

[50]:
```
array([0.8488177 , 0.17889592, 0.05436321])
```

It can be observed that the same random numbers are generated each time the code is run. This is useful for reproducibility of results, especially in scientific experiments or simulations where you want to ensure that the same random numbers are used across different runs of the code.

2.4 Task 1.7 Array Attributes and Methods

Here are some attributes and methods for an array:

[27]:
```
arr = np.arange(20)
ranarr = np.random.randint(0,100,10)
```

[28]:
```
arr
```

[28]:
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19])
```

[29]:
```
ranarr
```

[29]:
```
array([28, 27, 71, 75, 85, 47, 93, 17, 31, 23])
```

2.5 Reshape

In arr array we have 20 values. In this example, we will reshape 20 values in 4 rows and 5 columms. Returns an array containing the same data with a new shape. [reference — deprecated]

updated reference

```
numpy.reshape(a, /, shape, order='C', *, copy=None)
```

2.5.1 Parameters

- **a** — array-like

  Input array to reshape.

- **shape** — integer or tuple of integers

  Specifies the output dimensions, which must accommodate all input elements. An integer produces a one-dimensional array. One dimension may be `-1`, allowing NumPy to calculate it.

- **order** — `'C'`, `'F'`, or `'A'`, optional

  Sets the indexing order used to read and place elements:

  - `'C'`: last axis varies fastest; the default.
  - `'F'`: first axis varies fastest.
  - `'A'`: follows Fortran order for Fortran-contiguous input; otherwise, follows C order.

  This controls indexing, not the output's memory layout.

- **copy** — boolean or `None`, optional

  - `True`: always copies the data.
  - `None`: copies only when necessary; the default.
  - `False`: raises `ValueError` if reshaping requires copying.

2.5.2 Returns

An `ndarray` with the requested shape: a view when possible, otherwise a copy. Its memory layout is not guaranteed.

[30]:
```
arr.reshape(4,5)
```

[30]:
```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19]])
```

2.5.3 max, min, argmax, argmin

numpy.argmax: Returns the indices of the maximum values along an axis.

Remember that indexes start at 0 in Python, so the first element is at index 0, the second at index 1, and so on.

[31]:
```
ranarr
```

[31]:
```
array([28, 27, 71, 75, 85, 47, 93, 17, 31, 23])
```

[32]:
```
ranarr.max()
```

[32]:
```
93
```

[33]:
```
ranarr.argmax()
```

[33]:
```
6
```

[34]:
```
ranarr.min()
```

[34]:
```
17
```

[35]:
```
ranarr.argmin()
```

[35]:
```
7
```

2.6 Shape

Shape is an attribute that arrays have (not a method): [reference — deprecated]

updated reference

[36]:
```
# Vector
arr.shape
```

[36]:
```
(20,)
```

[37]:
```
# Notice the two sets of brackets
arr.reshape(2,10)
```

[37]:
```
array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14, 15, 16, 17, 18, 19]])
```

[38]:
```
arr.reshape(1,20).shape
```

[38]:
```
(1, 20)
```

[39]:
```
arr.reshape(10,2)
```

[39]:
```
array([[ 0,  1],
       [ 2,  3],
       [ 4,  5],
       [ 6,  7],
       [ 8,  9],
       [10, 11],
       [12, 13],
       [14, 15],
       [16, 17],
       [18, 19]])
```

[40]:
```
arr.reshape(10,2).shape
```

[40]:
```
(10, 2)
```

2.6.1 dtype

You can also grab the data type of the object in the array: [reference — deprecated]

updated reference

[41]:
```
arr.dtype
```

[41]:
```
dtype('int64')
```

[42]:
```
arr2 = np.array([2.5, 3.5, 4.5])
arr2.dtype
```

[42]:
```
dtype('float64')
```